provadis Hochschule

WAB

Provadis School of International Management and Technology

Performance of Various Python Runtimes for a Bioinformatics Algorithm

Testing the Smith-Waterman algorithm across CPython, PyPy, and Python 3.14's native JIT

Rubin Chempananickal James
rubin.chempananickal-james@stud-provadis-hochschule.de
Matriculation Number: D876

Department: Information Technology
Module: Fortgeschrittene Programmierung
Reviewer: Prof. Dr. Jörg Daubert

27.04.2026

Abstract

Write your abstract here.

Contents

Abstract II
List of Figures IV
List of Tables V
Glossary VI
Abbreviations VII
1. Introduction 1
    1.1. Background 1
        1.1.1. Cython 1
    1.2. Research Question 3
2. Methods 4
    2.1. Design 4
    2.2. Data Collection 4
References i
AI Declaration ii
Declaration of Authorship iv

List of Figures

Figure 1 Example of a local alignment between two DNA sequences, as produced by the Smith-Waterman algorithm. 3

List of Tables

Glossary

local alignment: An alignment strategy that searches for the best matching subsections of two sequences instead of forcing an end-to-end alignment. (pp. 2)

warm-up phase: The initial execution period in which a JIT-enabled runtime collects profiling information and may compile hot paths before reaching steady-state performance. (pp.)

Abbreviations

JIT, Just-in-Time (pp. 1, 2, 3)

PEP, Python Enhancement Proposal (pp. 1)

SW, Smith-Waterman (pp. 2, 3)

1. Introduction

1.1. Background

Python[1] has emerged to be the most widely used programming language among many fields, including bioinformatics[2]. It has many strengths, such as a very beginner friedly syntax, an exceptionally large ecosystem of packages, dynamic typing, and a feature rich standard library. One area where it is clearly lacking, however, is performance. This is due to the fact that the default implementation of Python, CPython, is interpreted and dynamically typed, which leads to significant overhead at runtime.

> The Python interpreter does a lot of work to try to abstract away the underlying computing elements that are being used. At no point does a programmer need to worry about allocating memory for arrays, how to arrange that memory, or in what sequence it is being sent to the CPU. This is a benefit of Python, since it lets you focus on the algorithms that are being implemented. However, it comes at a huge performance cost.
>
> — Gorelick M, Ozsvald I. [3]

Many solutions have been proposed to address this issue while still remaining within the Python ecosystem. Chief among them are:

- Transpilation into C using Cython[4]
- Tracing Just-in-Time (JIT) compilation, as implemented in an alternative Python interpreter to CPython, called PyPy[5]
- Copy-and-patch JIT compilation, as proposed in PEP 744[6], and with the latest stable implementation as of writing in Python 3.14[7].

1.1.1. Cython

Cython is a python module that allows users to write Python code interspersed with optional type annotations (in the form of `#cdef <variable>: <type>`). The file is then saved as a `.pyx` file and compiled into a native extension module using its *cythonize* toolchain and `setuptools`. In the end, it creates a compiled module (a `.pyd` file on Windows, `.so` on Linux) that can be imported and used from Python

just like any regular module. Cython also supports calling into C/C++ code and libraries, which makes it a very powerful tool for optimizing performance-critical sections of code while still maintaining the ease of use of Python.

Many other approaches exist, such as the .NET-based IronPython[8], the Python superset Mojo[9], etc., but due to the limited scope of this paper and the fact that these are either not as mature or as frequently maintained as the ones mentioned above, they will not be included in this comparison.

The native JIT approach is by far the newest, and it's still under active development and marked as experimental.

The Smith-Waterman (SW) algorithm[10] is a very common algorithm in bioinformatics. It is a local alignment algorithm that computes the highest-scoring matching subsequence(s) between two DNA/RNA/protein sequences (see Figure 1). It is a very computationally intensive algorithm, with a time complexity of O(m*n) for two sequences of length m and n, so a naive implementation in Python is expected to perform very poorly. The algorithm nevertheless has two nested "hot loops", which the performance-minded runtimes could potentially optimize. That, and the fact that it is still the best non-heuristic local alignment algorithm, and that it is relatively simple to implement (fewer than 100 lines of code), were the reasons why it was chosen as the algorithm for this performance comparison.

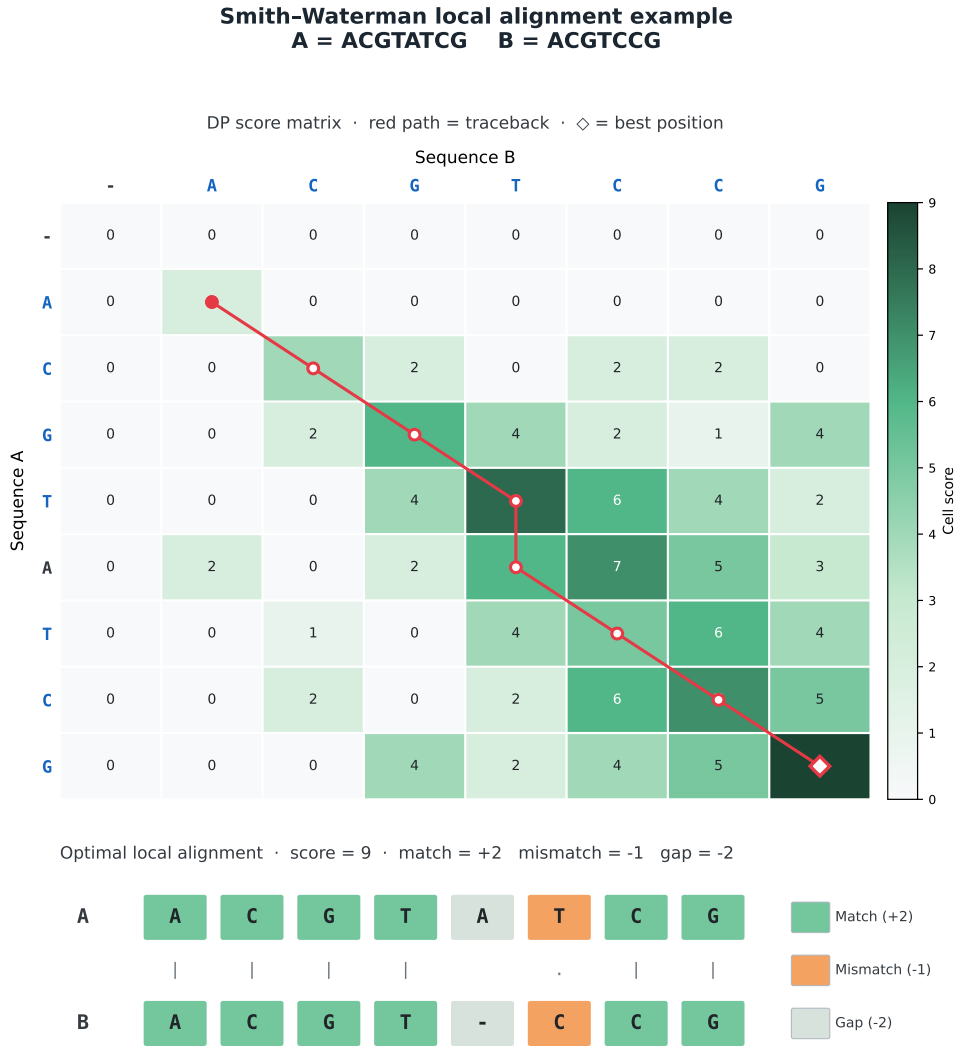

Figure 1: Example of a local alignment between two DNA sequences, as produced by the Smith-Waterman algorithm.

1.2. Research Question

How does the SW algorithm perform across the different Python runtimes, namely the default CPython interpreter, the newer CPython JIT, PyPy, and Cython?

2. Methods

Write your methods here.

2.1. Design

2.2. Data Collection

References

1. Van Rossum G, Drake FL. *Python 3 Reference Manual*. CreateSpace; 2009.
2. Mariano D, Ferreira M, Sousa BL, Santos LH, Melo-Minardi RC de. A Brief History of Bioinformatics Told by Data Visualization. In: Setubal JC, Silva WM, eds. *Advances in Bioinformatics and Computational Biology.* Springer International Publishing; 2020:235-246. doi:10.1007/978-3-030-65775-8_22
3. Gorelick M, Ozsvald I. *High Performance Python*. O'Reilly Media Inc.; 2025.
4. Behnel S, Bradshaw R, Citro C, Dalcin L, Seljebotn DS, Smith K. Cython: The Best of Both Worlds. *Computing in Science & Engineering.* 2011;13(2):31-39. doi:10.1109/MCSE.2010.118
5. The PyPy Team. PyPy Performance. 2026. Accessed March 25, 2026. https://www.pypy.org/performance.html
6. Bucher B, Ostrowski S. PEP 744 – JIT Compilation. 2024. Accessed March 10, 2026. https://peps.python.org/pep-0744/
7. Python Software Foundation. What's New In Python 3.14. 2025. Accessed March 10, 2026. https://docs.python.org/3/whatsnew/3.14.html
8. Foord M, Muirhead C. *Ironpython in Action*. Manning Publications Co.; 2009.
9. Mojo: Powerful CPU + GPU Programming. Accessed April 27, 2026. https://modular.com/mojo
10. Smith TF, Waterman MS. Identification of common molecular subsequences. *Journal of Molecular Biology.* 1981;147(1):195-197. doi:10.1016/0022-2836(81)90087-5

AI Declaration

The usage of AI tools within this project is documented here. I declare that I have documented all interactions with AI tools, including the prompts used and the outputs received.

System	Prompt	Usage
GitHub Copilot 1	I want you to rewrite my template from LaTeX to Typst, and make it look like it did before.	Recreated the template in Typst, maintaining the original appearance.
GitHub Copilot 2	I want you to write some tests Comparing the Python 3.14 JIT Compiler, PyPy, and Cython implementations of the Smith-Waterman algorithm.	Created a test suite that benchmarks the Python 3.14 JIT Compiler, PyPy, and Cython implementations of the Smith-Waterman algorithm.
GitHub Copilot 3	Put the test code in Code/, and use Python 3.15 instead of 3.14.	Added a runtime-generated Smith-Waterman validation and benchmarking suite in Code and retargeted the project text from Python 3.14 to Python 3.15.
GitHub Copilot 4	Okay, use python 3.14. 3.15 is still in pre-release. I've created the conda env named smith and installed the packages	Retargeted the Smith-Waterman suite and Exposé references back to Python 3.14 and validated the project against the configured environment.
GitHub Copilot 5	I think the warmup runs should be timed too. We can and should somehow separate it in the final summary though.	Modified the runtime worker to time each warmup repetition individually. Added a `per_run_times.csv` output that records every run's time with an `is_warmup` flag, keeping warmup and timed runs clearly separated in the data.

System	Prompt	Usage
GitHub Copilot 6	It'd also be interesting to see how the performance evolves across runs per algorithm, so make a plot of that.	Added a `run_convergence.pdf` plot showing runtime relative to each case's stable median across all run indices, with the warmup region shaded, faceted by problem size.
GitHub Copilot 7	The runtime distributions per case and algorithm would also be interesting to see, maybe as a box plot?	Added a `runtime_distributions.pdf` box plot showing the distribution of runtimes per case and algorithm, faceted by problem size.
GitHub Copilot 8	Add a power law fit of the runtime scaling with input size, and plot the empirical scaling exponent.	Added an empirical scaling exponent plot (`scaling_exponent.pdf`) fitting $t \propto n^{\alpha}$ via OLS in log-log space with 95% CI error bars, and a warmup penalty plot. The warmup penalty plot was subsequently removed for being too confusing to interpret.
GitHub Copilot 9	Move the run convergence plotting stuff to a new jupyter notebook. I might need to isolate that to singular scenarios too. Also move the scaling exponent plotting code in there.	Created `Code/plotting.ipynb` with interactive controls for both the run convergence and empirical scaling exponent plots. Both sections have filter controls for scenario, problem size, and algorithm, and the scaling exponent can be faceted per scenario to compare behaviour across input types.
GitHub Copilot 10	Help me create a python script that creates a figure to represent the Smith-Waterman algorithm's dynamic programming matrix and traceback path for a given pair of sequences.	Created `Code/swfigure.py`, which generates a PDF/PNG figure of the Smith-Waterman dynamic programming matrix.

Declaration of Authorship

I hereby confirm that I have personally and independently prepared the present work and have not used any sources or aids other than those specified. All passages taken verbatim or in substance from other sources are identified as such. The drawings, illustrations and tables in this work are created by me or have been provided with an appropriate source reference. This work has not been submitted by me to any other university in the same or similar form for the acquisition of an academic degree.

Frankfurt, 27.04.2026

Rubin Chempananickal James